



Ecole nationale des
Sciences Appliquées – Fès



Compte Rendu:

TP3 : K-means et CAH

Réaliser par :

❖ Abir Majdi

Encadré par :

Prof. AKKAD

Année universitaire : 2025/2026

Filière : GDNC2

Introduction

Dans le domaine de la data science, le clustering est une technique essentielle permettant de regrouper des données similaires sans connaissance préalable des classes. Ce TP porte sur l'étude et l'application de deux méthodes de classification non supervisée : **K-Means** et la **Classification Ascendante Hiérarchique (CAH)**.

À travers l'analyse de deux types de datasets (consommateurs et étudiants), ce travail vise à comprendre comment segmenter des individus selon leurs comportements ou leurs performances. L'objectif est également d'interpréter les résultats obtenus afin d'en tirer des conclusions pertinentes pour la prise de décision.

Objectifs

Ce TP a pour objectifs principaux :

- Comprendre le principe du **clustering** et son utilité en analyse de données
- Appliquer l'algorithme **K-Means** pour segmenter des données en différents groupes
- Mettre en œuvre la **classification hiérarchique agglomérative (CAH)** et analyser les dendrogrammes
- Identifier les variables pertinentes pour le clustering
- Interpréter les clusters obtenus dans différents contextes :
 - Analyse de comportement des consommateurs
 - Analyse des performances et de l'assiduité des étudiants
- Comparer les méthodes K-Means et CAH en termes de résultats et d'efficacité
- Utiliser des techniques comme la **standardisation** et le **score silhouette** pour améliorer la qualité du clustering

Partie 1 - Analyse d'un dataset de consommateurs

```
import pandas as pd
data = pd.read_excel("CustomerDataSet.xls")
data.shape
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie1+2.py"
(13, 5)
```

1. Afficher quelques attributs du dataset

```
# Afficher les premières lignes
print(data.head())

# Afficher les types des colonnes
print(data.dtypes)

# Statistiques descriptives
print(data.describe())
```

```
PS C:\Use> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et C
AH\partie1+2.py"
```

	Customer ID	ItemsBought	ItemsReturned	ZipCode	Product
0	4	45	10	2	1365
1	5	42	18	5	2764
2	6	50	0	1	1343
3	8	13	12	4	2435
4	9	10	7	3	2435

	Customer ID	ItemsBought	ItemsReturned	ZipCode	Product
Customer ID	int64				
ItemsBought	int64				
ItemsReturned	int64				
ZipCode	int64				
Product	int64				


```
dtype: object
```

	Customer ID	ItemsBought	ItemsReturned	ZipCode	Product
count	13.000000	13.000000	13.000000	13.000000	13.000000
mean	11.384615	34.461538	10.153846	4.000000	2063.000000
std	5.075077	14.181064	7.335373	2.915476	651.515925
min	4.000000	9.000000	0.000000	1.000000	1236.000000
25%	8.000000	34.000000	7.000000	2.000000	1365.000000
50%	11.000000	40.000000	9.000000	3.000000	2435.000000
75%	15.000000	42.000000	17.000000	6.000000	2465.000000
max	21.000000	50.000000	22.000000	9.000000	2896.000000

2. Quels sont les attributs qu'on peut utiliser pour le clustering ?

Les attributs numériques continus utilisables pour le clustering sont :

- ItemsBought : nombre d'articles achetés
- ItemsReturned : nombre d'articles retournés

On exclut :

- CustomerID : simple identifiant, sans signification pour le clustering
- ZipCode : code géographique catégoriel
- Product : identifiant de produit

```
print("Attributs utilisables pour le clustering : ItemsBought, ItemsReturned")
print(data[['ItemsBought', 'ItemsReturned']])
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie1+2.py"
Attributs utilisables pour le clustering : ItemsBought, ItemsReturned
```

	ItemsBought	ItemsReturned
0	45	10
1	42	18
2	50	0
3	13	12
4	10	7
5	34	17
6	40	20
7	40	8
8	9	9
9	36	7
10	42	1
11	46	1
12	41	22

3. Regrouper l'ensemble de données en utilisant K-Means.

```
# importer KMeans
from sklearn.cluster import KMeans

# importer matplotlib
import matplotlib.pyplot as plt

# importer preprocessing (StandardScaler pour la normalisation)
from sklearn import preprocessing

# créer le StandardScaler
scaler = preprocessing.StandardScaler()

# Copiez le dataframe avant le prétraitement afin que nous
# puissions accéder aux valeurs d'origine ultérieurement.
df2 = data.copy()

# préparer les deux attributs par le StandardScaler
df2[['ItemsBought', 'ItemsReturned']] = scaler.fit_transform(
    df2[['ItemsBought', 'ItemsReturned']]
)

# setup a figure
plt.figure(figsize=(14, 10))
ks = [2, 3, 4, 5, 6, 7]
```

```

# itérer sur la liste des valeurs de k
for idx, i in enumerate(ks, start=1):
    # créer le clusterer
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)

    # créer le regroupement
    labels = kmeans.fit_predict(df2[['ItemsBought', 'ItemsReturned']])

    # ajouter au subplot
    plt.subplot(3, 2, idx)
    plt.tight_layout(pad=3.0)

    # les étiquettes
    plt.title("#clusters (K) = {}".format(i))
    plt.xlabel('ItemsBought')
    plt.ylabel('ItemsReturned')

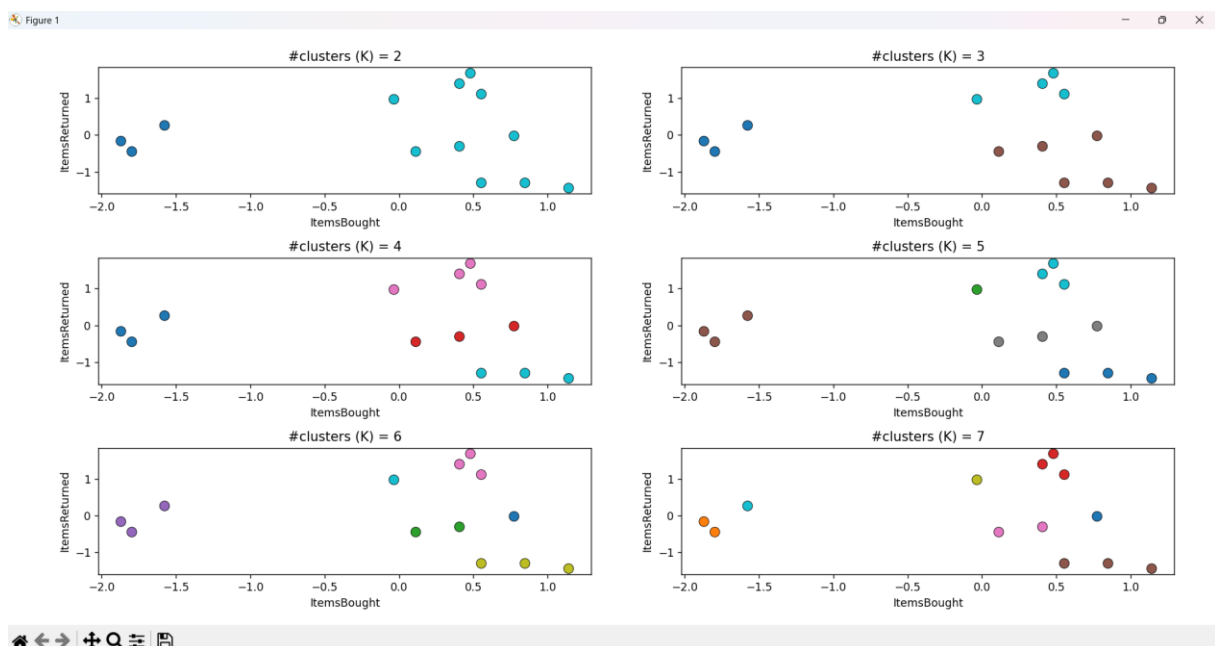
    # créer le scatter
    plt.scatter(df2['ItemsBought'], df2['ItemsReturned'],
                c=labels, cmap='tab10', s=80, edgecolors='k', linewidths=0.5)

# Afficher la figure
plt.show()

```

Pyth

df2.head()



5. Quelle(s) valeur(s) K ont du sens et comment étiquetez-vous les clusters résultants ?

K = 3 est la valeur la plus pertinente.

On peut clairement identifier 3 profils de clients :

Cluster 0 : Clients qui achètent beaucoup et retournent peu

- Bons clients fidèles

Cluster 1 : Clients avec achats et retours modérés

- Clients réguliers

Cluster 2 : Clients qui achètent peu mais retournent beaucoup

- Clients insatisfaits / à risque

```
print("Valeur K retenue : K = 3")
print("Cluster 0 : Bons clients fidèles (beaucoup d'achats, peu de retours)")
print("Cluster 1 : Clients réguliers (achats et retours moyens)")
print("Cluster 2 : Clients insatisfaits (peu d'achats, beaucoup de retours)")
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie1+2.py"
Valeur K retenue : K = 3
Cluster 0 : Bons clients fidèles (beaucoup d'achats, peu de retours)
Cluster 1 : Clients réguliers (achats et retours moyens)
Cluster 2 : Clients insatisfaits (peu d'achats, beaucoup de retours)
```

6. Ajouter une colonne contenant les étiquettes de classes à votre DataFrame

```
# Créer KMeans avec K = 3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)

# Ajouter la colonne 'Cluster' au DataFrame original
data['Cluster'] = kmeans3.fit_predict(df2[['ItemsBought', 'ItemsReturned']])

# Ajouter aussi à df2 (normalisé) pour les graphiques suivants
df2['Cluster'] = data['Cluster']

# Afficher le résultat
print(data[['Customer ID', 'ItemsBought', 'ItemsReturned', 'Cluster']])
```

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie1+2.py"
```

	Customer ID	ItemsBought	ItemsReturned	Cluster
0	4	45	10	1
1	5	42	18	2
2	6	50	0	1
3	8	13	12	0
4	9	10	7	0
5	10	34	17	2
6	11	40	20	2
7	12	40	8	1
8	14	9	9	0
9	15	36	7	1
10	16	42	1	1
11	17	46	1	1
12	21	41	22	2

7. Effectuez à nouveau le regroupement avec `K=3`. Ajoutez les identifiants de produits `Product` au graphique à l'aide de la fonction `annotate()` et interprétez les résultats.

```
# Créer KMeans avec K = 3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)

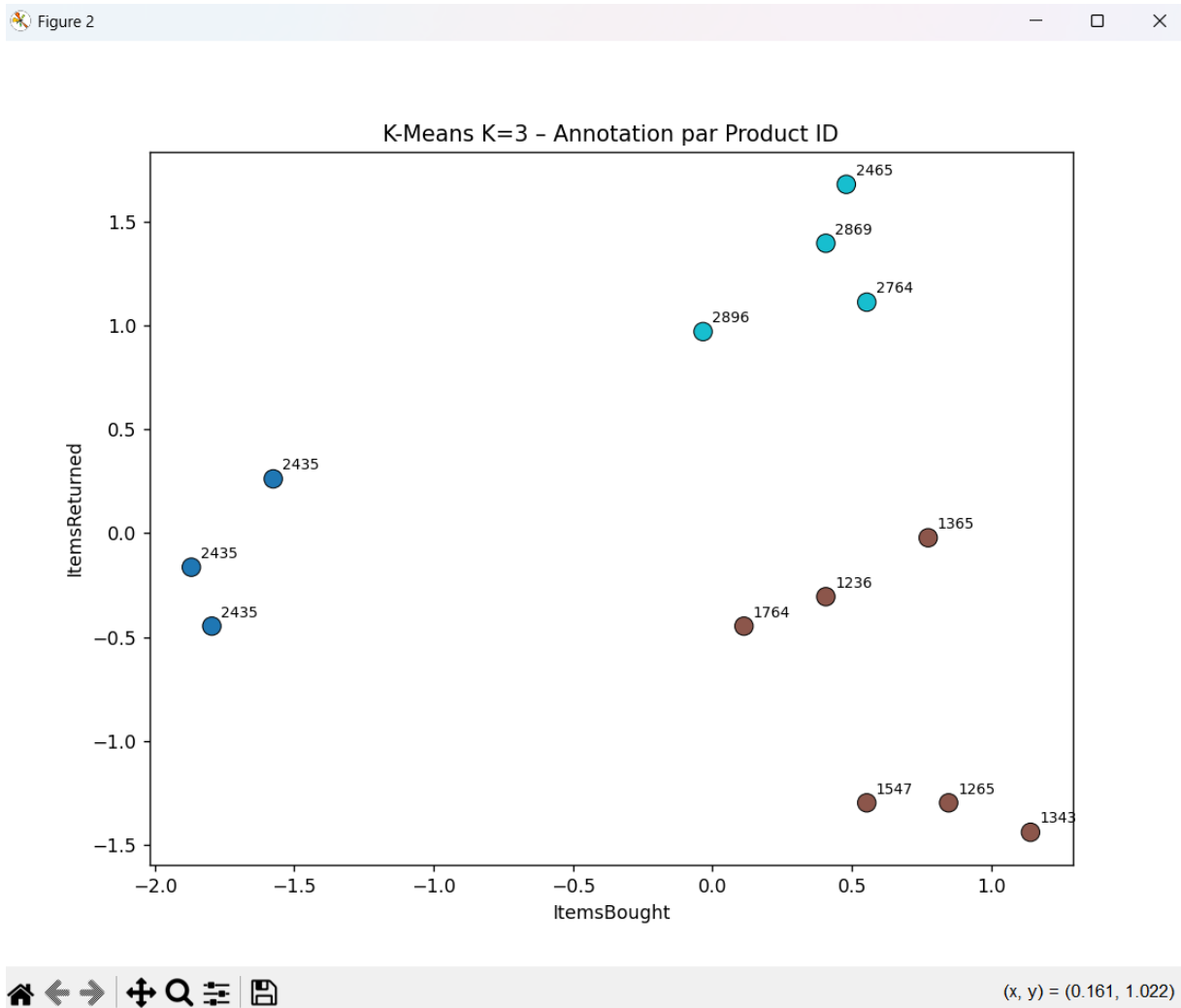
# Créer le clustering
labels = kmeans3.fit_predict(df2[['ItemsBought', 'ItemsReturned']])

# Créer un scatter
plt.figure(figsize=(9, 7))
plt.scatter(df2['ItemsBought'], df2['ItemsReturned'],
            c=labels, cmap='tab10', s=100, edgecolors='k', linewidths=0.6)

# annoter les produits en utilisant la colonne 'Product' et la fonction annotate
for i, row in data.iterrows():
    plt.annotate(str(row['Product']),
                (df2.loc[i, 'ItemsBought'], df2.loc[i, 'ItemsReturned']),
                textcoords="offset points", xytext=(5, 5), fontsize=8)

# les étiquettes des axes
plt.xlabel('ItemsBought')
plt.ylabel('ItemsReturned')
plt.title('K-Means K=3 □ Annotation par Product ID')

# Afficher
plt.show()
```



Interprétation :

- Les produits du cluster avec beaucoup d'achats et peu de retours (ex: 1343, 1365) sont des produits populaires et satisfaisants
- Les produits avec beaucoup de retours nécessitent une révision (qualité ou description produit à améliorer)

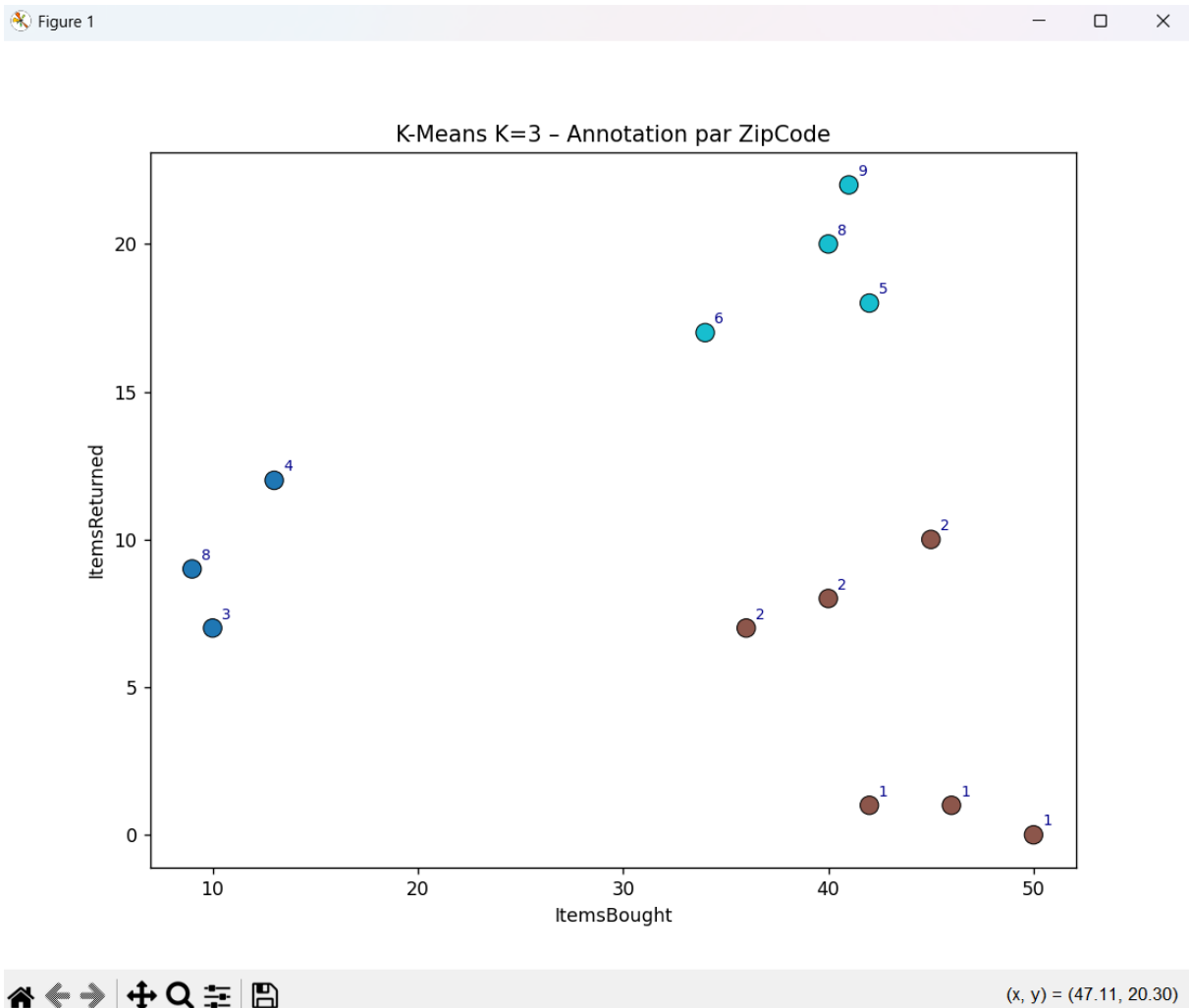
8. Interpréter le regroupement en utilisant le code zip.


```
# Créer le scatter (données originales non normalisées)
plt.figure(figsize=(9, 7))
plt.scatter(data['ItemsBought'], data['ItemsReturned'],
            c=data['Cluster'], cmap='tab10', s=100, edgecolors='k', linewidths=0.6)

# Annoter chaque point de données en utilisant le code ZIP
for i, row in data.iterrows():
    plt.annotate(str(row['ZipCode']),
                (row['ItemsBought'], row['ItemsReturned']),
                textcoords="offset points", xytext=(5, 5), fontsize=8, color='darkblue')

# les étiquettes
plt.xlabel('ItemsBought')
plt.ylabel('ItemsReturned')
plt.title('K-Means K=3 - Annotation par ZipCode')

# Afficher le plot
plt.show()
```



Interprétation :

- Les clients des zones ZipCode 1 et 2 achètent beaucoup et retournent peu

- Zones géographiques de bons clients

- Les clients du ZipCode 5 retournent davantage d'articles

- Peut indiquer une insatisfaction ou un problème de livraison dans cette région

9. Regroupez l'ensemble de données en utilisant la classification ascendante hiérarchique (Agglomerative Hierarchical Clustering).

Interprétation :

- Chaque fusion dans l'arbre représente un regroupement de clients similaires
- La hauteur d'une fusion indique la distance entre les groupes fusionnés
- Une grande hauteur = groupes très différents
- On peut couper l'arbre à différents niveaux pour obtenir N clusters

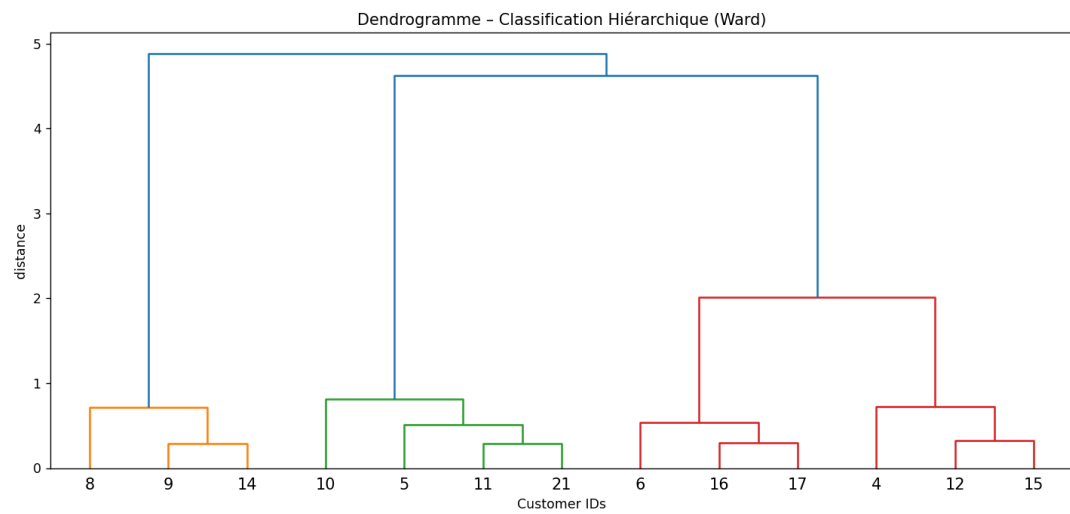
```
# importer linkage et dendrogram from scipy
from scipy.cluster.hierarchy import linkage, dendrogram

# Créer le clustering
Z = linkage(df2[['ItemsBought', 'ItemsReturned']], method='ward')

# Afficher le dendrogram
plt.figure(figsize=(14, 6))
dendrogram(Z, labels=data['Customer ID'].tolist())

# Les étiquettes
plt.xlabel('Customer IDs')
plt.ylabel('distance')
plt.title('Dendrogramme □ Classification Hiérarchique (Ward)')

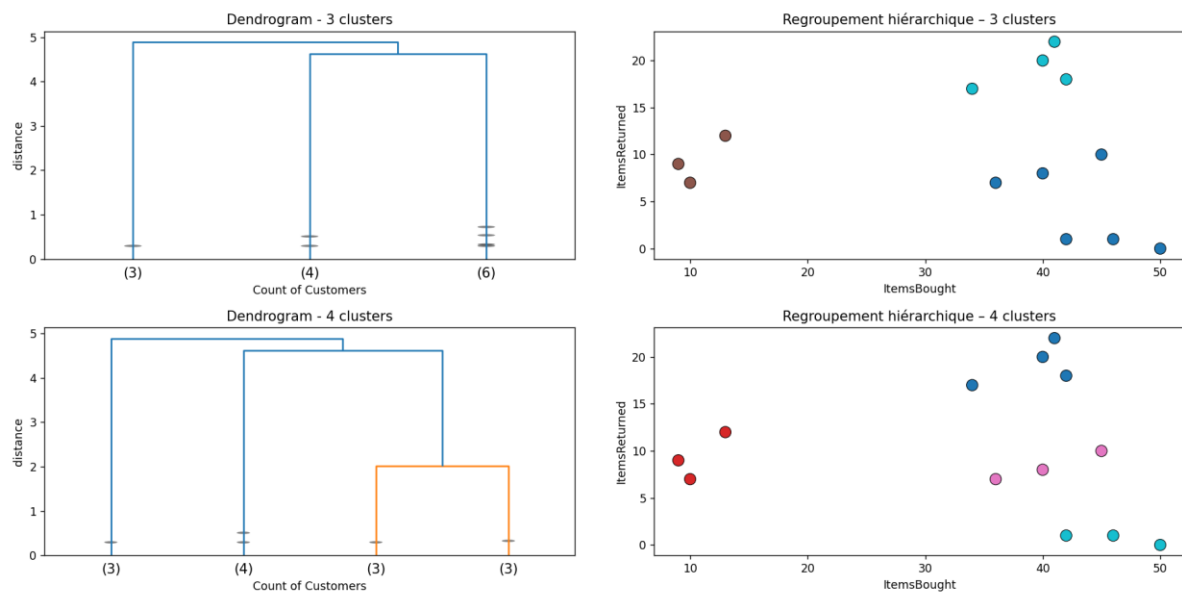
# Afficher
plt.show()
```



10. Tronquer la classification hiérarchique de manière à obtenir 3 ou 4 groupes de clients.

Interprétation :

- Avec 3 clusters : résultat cohérent avec K-Means ($K=3$), 3 profils distincts
- Avec 4 clusters : segmentation plus fine,
- un groupe se subdivise en deux sous-profils



```

from sklearn.cluster import AgglomerativeClustering

# setup a figure
plt.figure(figsize=(14, 10))
# itérer sur les différents nombres de clusters (ici: 3 et 4)
counter = 1
for i in [3, 4]:
    # ajouter un subplot
    plt.subplot(2, 2, counter)
    counter += 1
    # Les étiquettes
    plt.tight_layout(pad=3.0)
    plt.title('Dendrogram - {} clusters'.format(i))
    plt.xlabel('Count of Customers')
    plt.ylabel('distance')
    # Créer le clustering et afficher le dendrogram (tronqué)
    dendrogram(Z, truncate_mode='lastp', p=i, show_contracted=True)
    # Ajouter un second subplot (scatter)
    plt.subplot(2, 2, counter)
    counter += 1
    # Créer le clustering par troncature du dendrogram
    agg = AgglomerativeClustering(n_clusters=i)
    agg_labels = agg.fit_predict(df2[['ItemsBought', 'ItemsReturned']])
    plt.tight_layout(pad=3.0)
    plt.title('Regroupement hiérarchique [{} clusters'.format(i))
    plt.xlabel('ItemsBought')
    plt.ylabel('ItemsReturned')
    plt.scatter(data['ItemsBought'], data['ItemsReturned'],
                c=agg_labels, cmap='tab10', s=100, edgecolors='k', linewidths=0.6)
# Afficher la figure
plt.show()

```

Partie 2 : Analyse d'un dataset d'étudiant

1. Agréger le jeu de données des étudiants par étudiant et calculez la moyenne des notes ainsi que la moyenne du nombre de cours suivis. Utiliser la fonction `groupby()`

```

PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie1+2.py"

```

	Name	AvgMark	AvgAttended
0	Alex Krausche	1.325000	12.500000
1	Avid Morvita	3.100000	11.333333
2	Frank Fester	2.200000	11.600000
3	Mariano Selina	1.680000	6.200000
4	Michaela Martke	3.660000	7.400000
5	Otto Blacher	2.260000	11.800000
6	Steve Queck	2.800000	8.333333
7	Susanne Müller	2.433333	12.000000
8	Tanja Becker	1.666667	12.000000
9	Ulrich Gester	4.400000	8.600000

```
# Charger le fichier StudentData.xls dans un DataFrame
```

```
sdata = pd.read_excel("StudentData.xls")
```

```
# Afficher les premiers enregistrements
```

```
sdata.head()
```

Python

```
# Grouper le dataframe par étudiant (le nom de l'étudiant) et calculer les valeurs moyennes
```

```
student_agg = sdata.groupby('Name')[['Mark', 'Attended']].mean().reset_index()
```

```
student_agg.columns = ['Name', 'AvgMark', 'AvgAttended']
```

```
# Afficher les premiers enregistrements
```

```
student_agg.head(10)
```

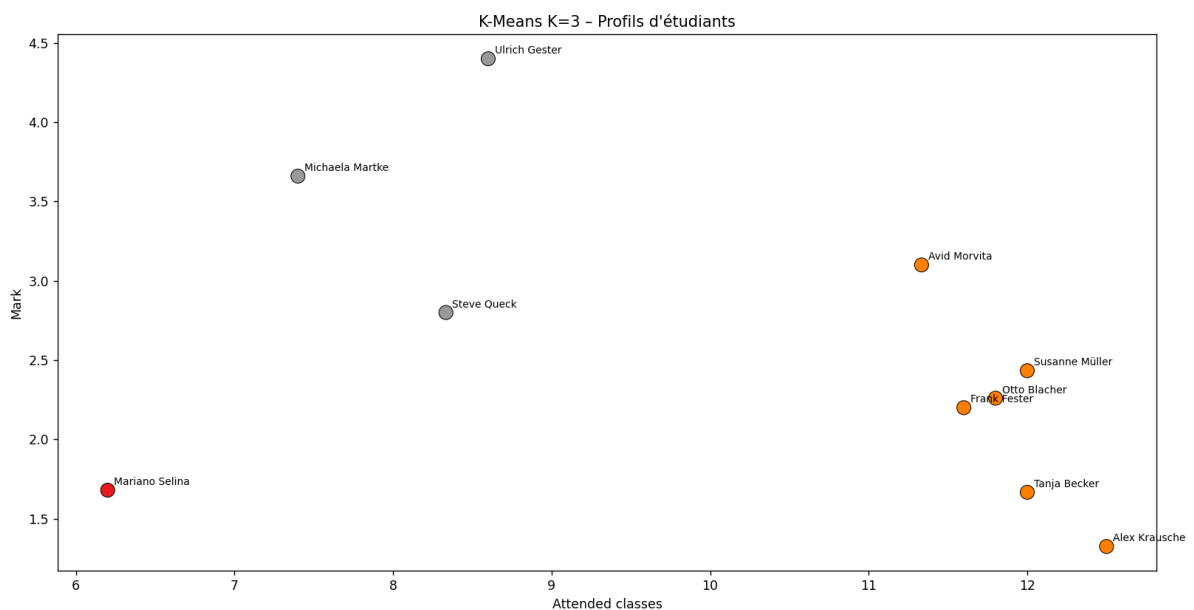
Python

```
print(student_agg)
```

2. Utiliser K-Means pour créer un regroupement.

Interprétation :

- Cluster 0 : Étudiants assidus avec bonnes notes → Étudiants performants
- Cluster 1 : Étudiants assidus mais notes moyennes → Étudiants réguliers
- Cluster 2 : Étudiants peu assidus et mauvaises notes → Étudiants en difficulté



```

# Envisager un prétraitement (normalisation)
scaler2 = preprocessing.StandardScaler()
X_students = scaler2.fit_transform(student_agg[['AvgAttended', 'AvgMark']])

# Créer une instance de KMeans
kmeans_s = KMeans(n_clusters=3, random_state=42, n_init=10)

# Créer le clustering
student_agg['Cluster'] = kmeans_s.fit_predict(X_students)

# Afficher le scatter
plt.figure(figsize=(10, 8))
plt.scatter(student_agg['AvgAttended'], student_agg['AvgMark'],
            c=student_agg['Cluster'], cmap='Set1',
            s=120, edgecolors='k', linewidths=0.6)

# Les étiquettes
plt.xlabel("Attended classes")
plt.ylabel("Mark")
plt.title("K-Means K=3 ▢ Profils d'étudiants")

# Annoter les données par les noms des étudiants
for _, row in student_agg.iterrows():
    plt.annotate(row['Name'],
                (row['AvgAttended'], row['AvgMark']),
                textcoords="offset points", xytext=(5, 4), fontsize=8)

# Afficher la figure
plt.show()

```

3. Regrouper le dataset en utilisant le clustering hiérarchique agglomératif. Utiliser différentes métriques de liaisons.

Interprétation :

- Single : sensible aux outliers, tend à créer des chaînes
- Average : compromis équilibré
- Complete : clusters plus compacts et homogènes → méthode recommandée ici

```

# lister les métriques à utiliser
modes = ['single', 'average', 'complete']

# créer une figure
plt.figure(figsize=(20, 5))
y_axis = None

# itérer sur la liste des métriques
for i, mode in enumerate(modes):

    # Créer un subplot
    y_axis = plt.subplot(1, 3, i + 1)

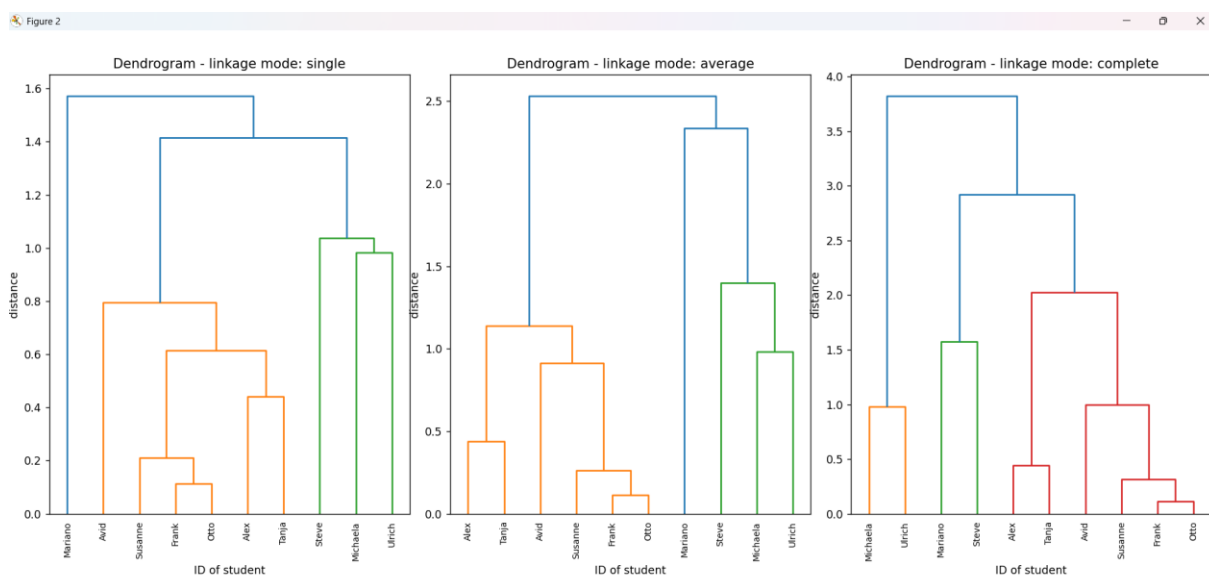
    # les étiquettes du subplot
    plt.title('Dendrogram - linkage mode: {}'.format(mode))
    plt.xlabel('ID of student')
    plt.ylabel('distance')

    # Créer le clustering
    Z_s = linkage(X_students, method=mode)

    # Afficher le dendrogram
    dendrogram(Z_s,
                labels=student_agg['Name'].apply(lambda x: x.split()[0]).tolist(),
                leaf_rotation=90,
                leaf_font_size=8)

# Afficher la figure
plt.tight_layout()
plt.show()

```



PARTIE 3 : Analyse des performances et de l'assiduité des étudiants

PARTIE 1 : Préparation des données

Exercice 1 : Agrégation des données

Objectif :

Regrouper les données par étudiant et calculer les moyennes.

Code :

```
import pandas as pd

# Charger le dataset
df = pd.read_excel("studentData.xls")

# Afficher les premières lignes
print(df.head())

# Agréger par étudiant
df_grouped = df.groupby("Name").agg({
    "Mark": "mean",
    "Attended": "mean"
}).reset_index()

# Renommer les colonnes
df_grouped.columns = ["Name", "Avg_Mark", "Avg_Attendance"]

print(df_grouped)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
Semester      Name      Course  Mark  Attended
0  FSS2010  Alex Krausche  Database Systems I    1.3      13
1  FSS2010   Tanja Becker  Database Systems I    2.0      12
2  FSS2010  Mariano Selina  Database Systems I    1.7       5
3  FSS2010   Otto Blacher  Database Systems I    2.3      13
4  FSS2010   Frank Fester  Database Systems I    2.0      13
   Name  Avg_Mark  Avg_Attendance
0  Alex Krausche  1.325000    12.500000
1   Avid Morvita  3.100000    11.333333
2   Frank Fester  2.200000    11.600000
3  Mariano Selina  1.680000     6.200000
4 Michaela Martke  3.660000     7.400000
5   Otto Blacher  2.260000    11.800000
6   Steve Queck  2.800000     8.333333
7  Susanne Müller  2.433333    12.000000
8   Tanja Becker  1.666667    12.000000
9  Ulrich Gester  4.400000     8.600000
```


PARTIE 2 : Préparation pour clustering

Exercice 2 : Standardisation

Objectif :

Normaliser les données pour éviter le biais d'échelle.

Code :

```
from sklearn.preprocessing import StandardScaler

# Sélection des variables
X = df_grouped[["Avg_Mark", "Avg_Attendance"]]

# Standardisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(X_scaled)
```

Output :

```
PS C:\Users> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
[[-1.34918206  1.06654124]
 [ 0.60177367  0.53097534]
 [-0.38744332  0.65339004]
 [-0.95899091 -1.82551463]
 [ 1.21728646 -1.27464685]
 [-0.32149552  0.7452017 ]
 [ 0.27203467 -0.84619413]
 [-0.13097966  0.837013 ]
 [-0.97364598  0.837013 ]
 [ 2.03064265 -0.72377906]]
```

PARTIE 3 : K-Means

Exercice 3 : Clustering K-Means

Objectif :

Créer des groupes d'étudiants.

```

from sklearn.cluster import KMeans

# Choisir K=3
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

# Ajouter au dataset
df_grouped["Cluster"] = clusters

print(df_grouped)

```

Output :

```

PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"

```

	Name	Avg_Mark	Avg_Attendance	Cluster
0	Alex Krausche	1.325000	12.500000	1
1	Avid Morvita	3.100000	11.333333	1
2	Frank Fester	2.200000	11.600000	1
3	Mariano Selina	1.680000	6.200000	0
4	Michaela Martke	3.660000	7.400000	2
5	Otto Blacher	2.260000	11.800000	1
6	Steve Queck	2.800000	8.333333	2
7	Susanne Müller	2.433333	12.000000	1
8	Tanja Becker	1.666667	12.000000	1
9	Ulrich Gester	4.400000	8.600000	2

Exercice 3 - Visualisation

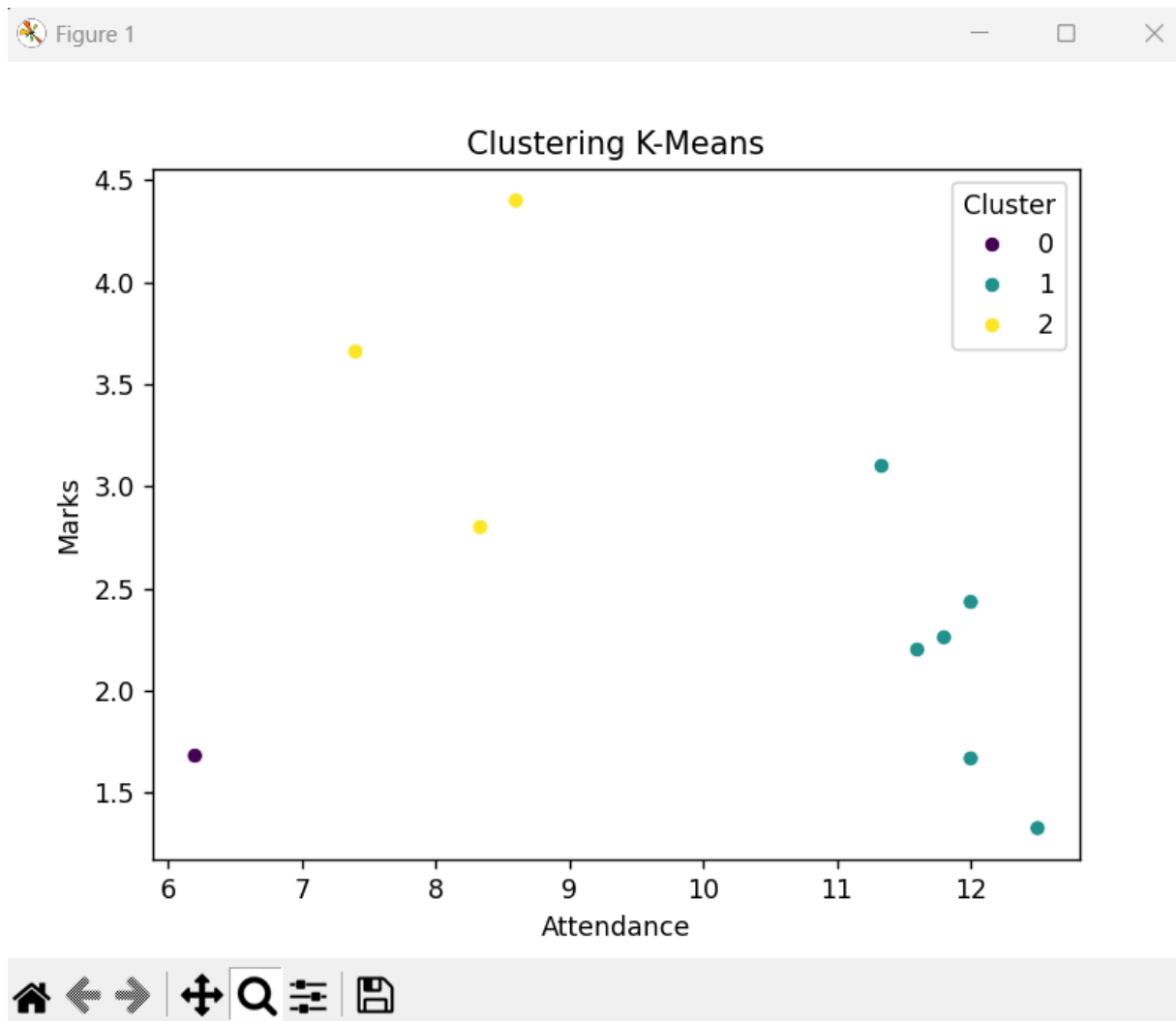
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure()
sns.scatterplot(
    x=df_grouped["Avg_Attendance"],
    y=df_grouped["Avg_Mark"],
    hue=df_grouped["Cluster"],
    palette="viridis"
)

plt.title("Clustering K-Means")
plt.xlabel("Attendance")
plt.ylabel("Marks")

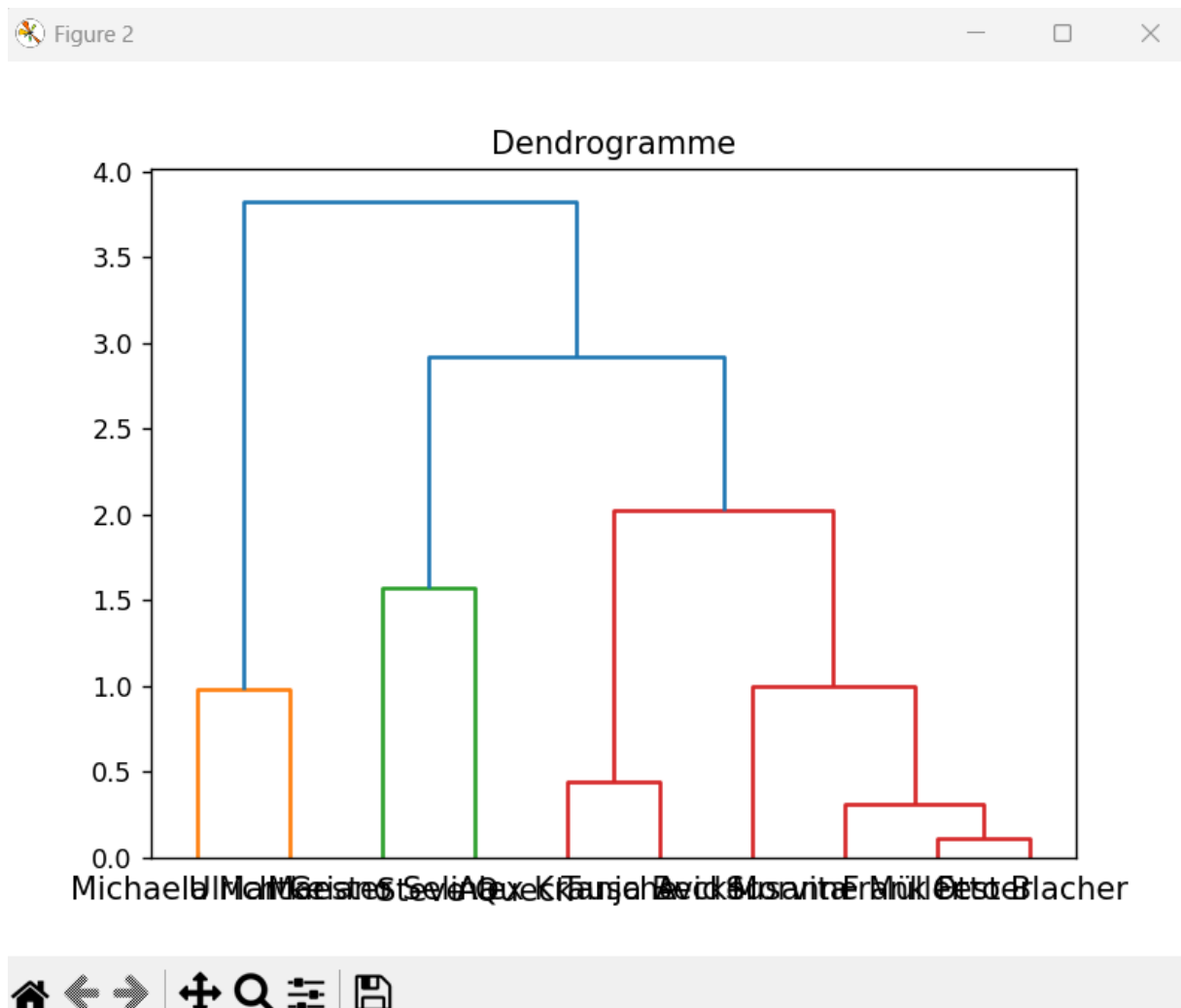
plt.show()
```

Output :



PARTIE 4 : Clustering hiérarchique

Exercice 4 : Dendrogramme



PARTIE 5 : QUESTIONS

Question 1 : Meilleur étudiant

```
best_student = df_grouped.loc[df_grouped["Avg_Mark"].idxmax()]
print("Meilleur étudiant :", best_student)
```

Output :

```
Meilleur étudiant : Name      Ulrich Gester
Avg_Mark              4.4
Avg_Attendance        8.6
Cluster              2
Name: 9, dtype: object
```

Question 2 : Plus assidu

```
most_attended = df_grouped.loc[df_grouped["Avg_Attendance"].idxmax()]
print("Plus assidu :", most_attended)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
Plus assidu : Name      Alex Krausche
Avg_Mark      1.325
Avg_Attendance 12.5
Cluster      1
Name: 0, dtype: object
```

Question 3 : Plus faible

```
worst_student = df_grouped.loc[
    (df_grouped["Avg_Mark"] + df_grouped["Avg_Attendance"]).idxmin()
]
print("Plus faible :", worst_student)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
Plus faible : Name      Mariano Selina
Avg_Mark      1.68
Avg_Attendance 6.2
Cluster      0
Name: 3, dtype: object
```

Question 4 : Corrélation

```
correlation = df_grouped["Avg_Mark"].corr(df_grouped["Avg_Attendance"])
print("Corrélation :", correlation)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
Corrélation : -0.40376372794103255
```

Interprétation :

- proche de 1 → forte corrélation
- proche de 0 → aucune relation

PARTIE 6 : Clustering avancé

Question 5 : Tester plusieurs K

```
for k in [2, 3, 4, 5]:
    kmeans = KMeans(n_clusters=k, random_state=42)
    df_grouped[f"Cluster_{k}"] = kmeans.fit_predict(X_scaled)

print(df_grouped)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
      Name Avg_Mark Avg_Attendance ... Cluster_3 Cluster_4 Cluster_5
0  Alex Krausche  1.325000      12.500000 ...         1         1         1
1   Avid Morvita  3.100000      11.333333 ...         1         3         3
2   Frank Fester  2.200000      11.600000 ...         1         1         1
3  Mariano Selina  1.680000       6.200000 ...         0         0         0
4  Michaela Martke  3.660000       7.400000 ...         2         2         2
5   Otto Blacher  2.260000      11.800000 ...         1         1         1
6   Steve Queck  2.800000       8.333333 ...         2         3         3
7  Susanne Müller  2.433333      12.000000 ...         1         1         1
8   Tanja Becker  1.666667      12.000000 ...         1         1         1
9   Ulrich Gester  4.400000       8.600000 ...         2         2         4

[10 rows x 8 columns]
```

Question 6 : Score silhouette

```
from sklearn.metrics import silhouette_score

for k in [2, 3, 4, 5]:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    print(f"K={k}, Silhouette Score={score}")
```

Le meilleur K = score le plus élevé

Question 7 : Points frontières

```
import numpy as np

distances = kmeans.transform(X_scaled)
df_grouped["Min_Distance"] = np.min(distances, axis=1)

# Les plus proches des frontières
boundary_students = df_grouped.sort_values("Min_Distance").head()
print(boundary_students)
```

Output :

```
PS C:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
      Name Avg_Mark Avg_Attendance ... Cluster_4 Cluster_5 Min_Distance
3  Mariano Selina   1.68         6.2 ...         0         0   0.000000
4  Michaela Martke   3.66         7.4 ...         2         2   0.000000
9   Ulrich Gester   4.40         8.6 ...         2         4   0.000000
2    Frank Fester   2.20        11.6 ...         1         1   0.300843
5    Otto Blacher   2.26        11.8 ...         1         1   0.321842

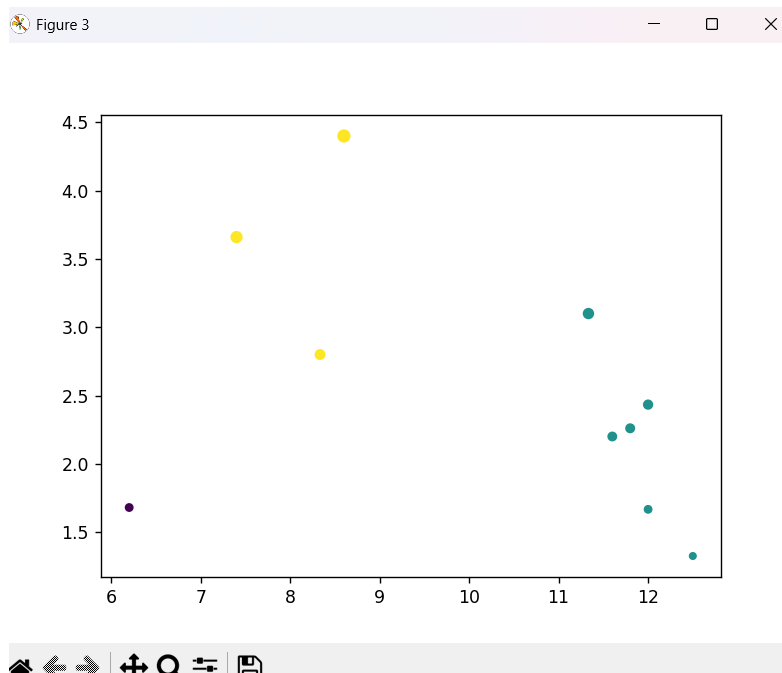
[5 rows x 9 columns]
```

PARTIE 7 : Visualisation avancée

Question 8 : Taille selon note

```
plt.figure()
plt.scatter(
    df_grouped["Avg_Attendance"],
    df_grouped["Avg_Mark"],
    s=df_grouped["Avg_Mark"] * 10,
    c=df_grouped["Cluster"],
    cmap="viridis"
)
plt.show()
```

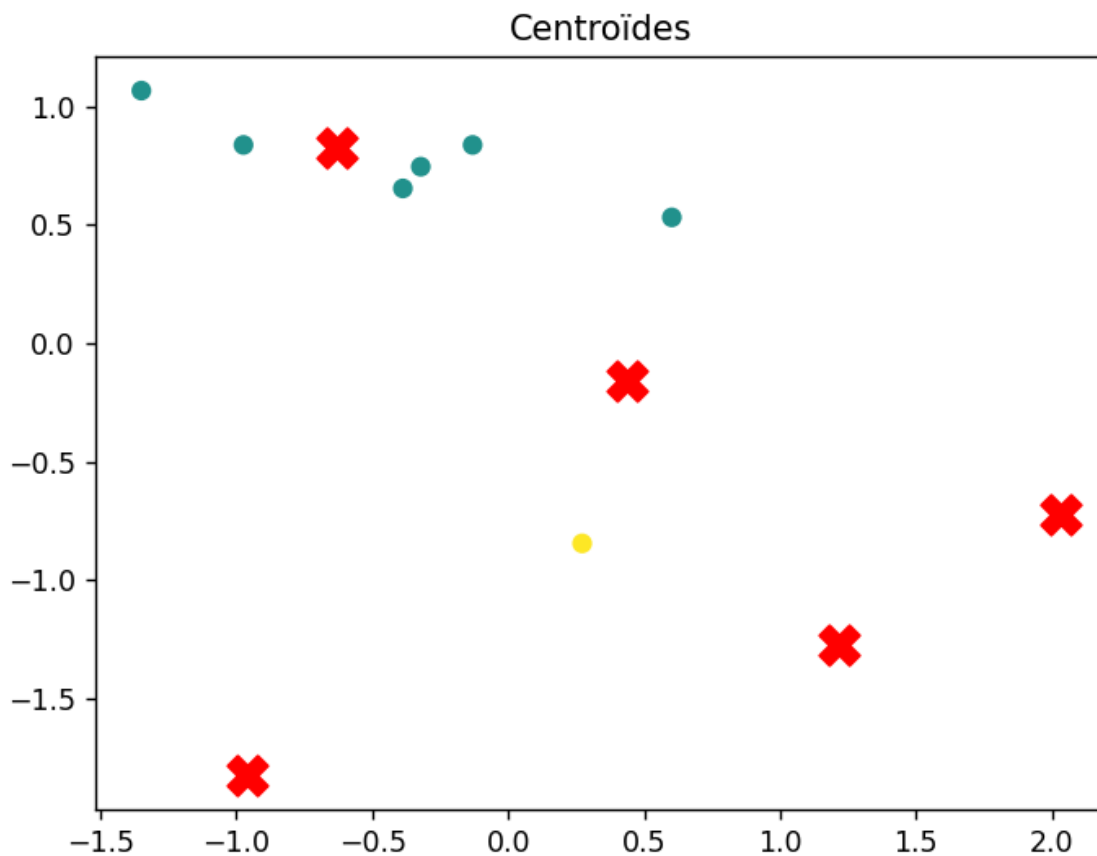
Output :



Question 9 : Centroides

```
centroids = kmeans.cluster_centers_  
  
plt.figure()  
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters)  
  
plt.scatter(  
    centroids[:, 0],  
    centroids[:, 1],  
    c='red',  
    marker='x',  
    s=200  
)  
  
plt.title("Centroides")  
plt.show()
```

Output :



Question 10 : Comparaison

- K-Means → rapide, clusters sphériques
- Hiérarchique → plus détaillé, dendrogramme

PARTIE 8 : Analyse (RÉPONSES À RÉDIGER)

Question 11 : Stratégies

Exemple :

- Cluster 1 : bons étudiants → mentorat
- Cluster 2 : manque d'assiduité → suivi
- Cluster 3 : faibles → soutien intensif

Question 12 : Tuteurs

```
tutors = df_grouped[df_grouped["Cluster"] == 0]
print(tutors)
```

Output :

```
PS C:\Users> python -u "c:\Users\saidm\Documents\IA mercredi\TP 3 K-means et CAH\TP 3 K-means et CAH\partie3.py"
      Name  Avg_Mark  Avg_Attendance  ...  Cluster_4  Cluster_5  Min_Distance
3  Mariano Selina    1.68           6.2  ...         0         0           0.0
[1 rows x 9 columns]
```

Question 13 : Amélioration

- Participation
- Projets
- Retards
- Activités extra

Conclusion

Ce TP nous a permis de découvrir concrètement les méthodes de clustering et leur importance dans l'analyse exploratoire des données. L'utilisation de **K-Means** a montré son efficacité pour segmenter rapidement les données en groupes distincts, tandis que la **classification hiérarchique** a offert une vision plus détaillée des relations entre les individus.

L'analyse des datasets a permis d'identifier différents profils, que ce soit des clients (fidèles, réguliers, à risque) ou des étudiants (performants, moyens, en difficulté). Ces résultats peuvent être

exploités pour adapter des stratégies spécifiques, comme améliorer la satisfaction client ou accompagner les étudiants en difficulté.

En conclusion, ce TP met en évidence l'importance du choix des méthodes et des paramètres dans le clustering, ainsi que la nécessité d'interpréter correctement les résultats pour en tirer des décisions pertinentes.